

React POS System Lab: App Router & State Lifting

MERN Stack Bootcamp – Hands-On Practice

Instructor: Kashif Raza | [LinkedIn](#)

Duration: ~3 hours

Difficulty: Beginner → Intermediate



Lab Overview

You will build a **client-side Point of Sale (POS) kiosk** using Next.js App Router. This lab intentionally **excludes the Context API** to help you experience prop drilling, state lifting, and `localStorage` synchronization firsthand. You will apply:

- **Next.js App Router** for file-based navigation
- `useState` for local UI state & cart tracking
- `useEffect` for side effects (persistence, validation, totals)
- **Prop drilling** to share cart state across components
- **Professional folder structure** used in production teams



Why No Context Yet?

Context is powerful, but beginners often overuse it. By building this lab with props + `localStorage`, you'll deeply understand *why* Context exists and how it simplifies the exact patterns you're writing today. We'll refactor this to Context in tomorrow's lecture.

Setup & Dependencies

```
# 1. Create Next.js project
npx create-next-app@latest pos-lab --js --tailwind --eslint --no-app-dir=false
cd pos-lab

# 2. Install icons (optional but recommended)
npm install lucide-react
```

Verify `next.config.mjs` enables App Router:

```
const nextConfig = { reactStrictMode: true };
export default nextConfig;
```

Professional Folder Structure

Create this exact structure. It separates concerns cleanly and scales well.

```
pos-lab/
├── app/
│   ├── layout.jsx           # Root wrapper + metadata
│   ├── page.jsx             # Main POS screen (Catalog + Cart)
│   ├── checkout/
│   │   ├── page.jsx        # Checkout form & order summary
│   │   └── receipt/
│   │       └── [id]/
│   │           └── page.jsx  # Dynamic receipt confirmation
├── components/
│   ├── ProductGrid.jsx
│   ├── ProductCard.jsx
│   ├── CartPanel.jsx
│   ├── CartItem.jsx
│   └── SearchBar.jsx
├── hooks/
│   └── useCart.js           # Custom hook: useState + useEffect + localStorage
├── lib/
│   └── products.js          # Mock product database
├── public/
└── package.json
```

◆ Phase 1: Mock Data & Custom Cart Hook (45 mins)

Step 1.1: Mock Products (lib/products.js)

```
export const products = [
  { id: 'p1', name: 'Wireless Mouse', price: 29.99, category: 'accessories' },
  { id: 'p2', name: 'Mech Keyboard', price: 89.50, category: 'accessories' },
  { id: 'p3', name: 'USB-C Hub', price: 45.00, category: 'accessories' },
  { id: 'p4', name: 'Monitor Stand', price: 34.99, category: 'furniture' },
  { id: 'p5', name: 'Webcam 1080p', price: 59.99, category: 'accessories' },
  { id: 'p6', name: 'Desk Lamp', price: 24.50, category: 'lighting' },
];
```

Step 1.2: Custom useCart Hook (hooks/useCart.js)

```
import { useState, useEffect } from 'react';

export function useCart() {
  const [items, setItems] = useState([]);

  // Load from localStorage on mount
  useEffect(() => {
    try {
      const saved = localStorage.getItem('pos-cart');
      if (saved) setItems(JSON.parse(saved));
    } catch (e) { console.error('Failed to load cart'); }
  }, []);

  // Sync to localStorage on change
  useEffect(() => {
    localStorage.setItem('pos-cart', JSON.stringify(items));
  }, [items]);

  const addItem = (product) => {
    setItems(prev => {
      const existing = prev.find(i => i.id === product.id);
      if (existing) {
        return prev.map(i => i.id === product.id ? { ...i, quantity: i.quantity + 1 } : i);
      }
      return [ ...prev, { ...product, quantity: 1 } ];
    });
  };

  const updateQty = (id, qty) => {
    if (qty <= 0) return setItems(prev => prev.filter(i => i.id !== id));
    setItems(prev => prev.map(i => i.id === id ? { ...i, quantity: qty } : i));
  };

  const clearCart = () => setItems([]);

  const total = items.reduce((sum, i) => sum + i.price * i.quantity, 0);
  const itemCount = items.reduce((sum, i) => sum + i.quantity, 0);

  return { items, total, itemCount, addItem, updateQty, clearCart };
}
```

✓ Success Criteria:

- ☐ Hook uses `useState` for cart array

- ☐ `useEffect` handles mount load & change persistence
- ☐ All functions return new arrays (immutability)
- ☐ Derived state (`total` , `itemCount`) updates automatically

◆ Phase 2: Routing & Main POS Layout (45 mins)

Step 2.1: Root Layout (app/layout.jsx)

```
import './globals.css';

export const metadata = {
  title: 'QuickPOS | MERN Bootcamp',
  description: 'Client-side POS system lab'
};

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body className="bg-gray-100 min-h-screen font-sans">
        <header className="bg-indigo-700 text-white p-4 shadow">
          <div className="max-w-7xl mx-auto flex justify-between items-center">
            <h1 className="text-xl font-bold">🛒 QuickPOS</h1>
            <span className="text-sm opacity-80">State Lifting Lab</span>
          </div>
        </header>
        <main className="max-w-7xl mx-auto p-4">{children}</main>
      </body>
    </html>
  );
}
```

Step 2.2: Main POS Screen (app/page.jsx)

```
'use client';
import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { products } from '@lib/products';
import { useCart } from '@hooks/useCart';
import SearchBar from '@components/SearchBar';
import ProductGrid from '@components/ProductGrid';
import CartPanel from '@components/CartPanel';

export default function Home() {
  const cart = useCart();
  const router = useRouter();
  const [search, setSearch] = useState('');
  const [filtered, setFiltered] = useState(products);

  // Filter products reactively
  useEffect(() => {
    const q = search.toLowerCase();
    setFiltered(products.filter(p =>
      p.name.toLowerCase().includes(q) || p.category.includes(q)
    ));
  }, [search]);

  const handleCheckout = () => {
    if (cart.itemCount === 0) return alert('Cart is empty!');
    router.push('/checkout');
  };

  return (
    <div className="grid md:grid-cols-3 gap-6">
      <div className="md:col-span-2 space-y-4">
```

```
    <SearchBar value={search} onChange={setSearch} />
    <ProductGrid products={filtered} onAdd={cart.addItem} />
  </div>
  <div className="md:col-span-1">
    <CartPanel
      cart={cart}
      onCheckout={handleCheckout}
      onUpdateQty={cart.updateQty}
      onRemove={cart.updateQty} // qty 0 removes
    />
  </div>
</div>
);
}
```

✅ **Success Criteria:**

- ☐ `useState` manages search input
- ☐ `useEffect` filters products without mutating original array
- ☐ Cart state initialized via custom hook
- ☐ Props passed cleanly to child components

◆ Phase 3: Component Implementation (60 mins)

components/SearchBar.jsx

```
export default function SearchBar({ value, onChange }) {
  return (
    <input
      type="text"
      placeholder="🔍 Search products or categories..."
      value={value}
      onChange={(e) => onChange(e.target.value)}
      className="w-full p-3 border rounded shadow-sm focus:outline-none focus:ring-2 focus:ring-indigo-500"
    />
  );
}
```

components/ProductGrid.jsx & components/ProductCard.jsx

```
// ProductGrid.jsx
export default function ProductGrid({ products, onAdd }) {
  return (
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
      {products.map(p => <ProductCard key={p.id} product={p} onAdd={onAdd} />)}
    </div>
  );
}

// ProductCard.jsx
export default function ProductCard({ product, onAdd }) {
  return (
    <div className="bg-white p-4 rounded shadow hover:shadow-md transition">
      <div className="h-24 bg-gray-200 rounded mb-3 flex items-center justify-center text-2xl">🍪
    </div>
    <h3 className="font-semibold text-sm">{product.name}</h3>
    <p className="text-xs text-gray-500 capitalize">{product.category}</p>
    <div className="flex justify-between items-center mt-3">
      <span className="font-bold text-indigo-600">${product.price.toFixed(2)}</span>
      <button
        onClick={() => onAdd(product)}
        className="bg-indigo-600 text-white px-3 py-1 rounded text-xs hover:bg-indigo-700"
      >
        + Add
      </button>
    </div>
  </div>
  );
}
```

components/CartPanel.jsx & components/CartItem.jsx

```
// CartPanel.jsx
import CartItem from './CartItem';

export default function CartPanel({ cart, onCheckout, onUpdateQty, onRemove }) {
  if (cart.items.length === 0) {
    return <div className="bg-white p-6 rounded shadow text-center text-gray-500">Cart is empty</div>;
  }
}
```

```

return (
  <div className="bg-white p-4 rounded shadow flex flex-col h-full">
    <h2 className="font-bold text-lg mb-3">Current Order</h2>
    <div className="flex-1 overflow-y-auto max-h-64 space-y-2 mb-4">
      {cart.items.map(item => (
        <CartItem
          key={item.id}
          item={item}
          onQtyChange={onUpdateQty}
          onRemove={onRemove}
        />
      ))}
    </div>
    <div className="border-t pt-3">
      <div className="flex justify-between font-bold text-lg mb-3">
        <span>Total</span>
        <span>${cart.total.toFixed(2)}</span>
      </div>
      <button
        onClick={onCheckout}
        className="w-full bg-green-600 text-white py-2 rounded hover:bg-green-700 transition"
      >
        Checkout ({cart.itemCount} items)
      </button>
    </div>
  </div>
);
}

// CartItem.jsx
export default function CartItem({ item, onQtyChange, onRemove }) {
  return (
    <div className="flex justify-between items-center border-b pb-2">
      <div className="flex-1">
        <p className="font-medium text-sm truncate">{item.name}</p>
        <p className="text-xs text-gray-500">${item.price} × {item.quantity}</p>
      </div>
      <div className="flex items-center gap-2">
        <button onClick={() => onQtyChange(item.id, item.quantity - 1)} className="w-6 h-6 bg-gray-200 rounded text-sm">-</button>
        <span className="text-sm w-4 text-center">{item.quantity}</span>
        <button onClick={() => onQtyChange(item.id, item.quantity + 1)} className="w-6 h-6 bg-gray-200 rounded text-sm">+</button>
        <button onClick={() => onRemove(item.id, 0)} className="ml-1 text-red-500">x</button>
      </div>
    </div>
  );
}

```

✓ Success Criteria:

- ☐ Components receive data & functions via props only
- ☐ Cart updates instantly without page reload
- ☐ localStorage syncs across refreshes
- ☐ Clean, responsive UI layout

◆ Phase 4: Checkout & Dynamic Receipt (45 mins)

Step 4.1: Checkout Page (app/checkout/page.jsx)

```
'use client';
import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { useCart } from '@hooks/useCart';

export default function CheckoutPage() {
  const cart = useCart();
  const router = useRouter();
  const [form, setForm] = useState({ name: '', email: '', phone: '' });
  const [isValid, setIsValid] = useState(false);
  const [processing, setProcessing] = useState(false);

  useEffect(() => {
    // Redirect if cart empty
    if (cart.itemCount === 0) router.push('/');

    // Validate on change
    setIsValid(
      form.name.trim().length >= 3 &&
      form.email.includes('@') &&
      form.phone.trim().length >= 10
    );
  }, [form, cart.itemCount, router]);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!isValid) return;
    setProcessing(true);

    // Simulate backend delay
    await new Promise(res => setTimeout(res, 1500));

    const orderId = `POS-${Date.now().toString(36).toUpperCase}`;
    cart.clearCart();
    router.push(`/receipt/${orderId}`);
  };

  return (
    <div className="max-w-lg mx-auto bg-white p-6 rounded shadow">
      <h2 className="text-xl font-bold mb-4">Complete Order</h2>
      <form onSubmit={handleSubmit} className="space-y-4">
        <input placeholder="Customer Name" value={form.name} onChange={e => setForm({ ...form, name: e.target.value})} className="w-full p-2 border rounded" />
        <input placeholder="Email" type="email" value={form.email} onChange={e => setForm({ ...form, email: e.target.value})} className="w-full p-2 border rounded" />
        <input placeholder="Phone Number" value={form.phone} onChange={e => setForm({ ...form, phone: e.target.value})} className="w-full p-2 border rounded" />

        <div className="bg-gray-50 p-3 rounded">
          <p className="font-semibold">Order Total: <span className="text-green-600">${cart.total.toFixed(2)}</span></p>
        </div>

        <button disabled={!isValid || processing} className="w-full bg-indigo-600 text-white py-2 rounded hover:bg-indigo-700 disabled:opacity-50">
          {processing ? 'Processing...' : 'Confirm Payment'}
        </button>
      </form>
    </div>
  );
}
```

```
    </form>
  </div>
);
}
```

Step 4.2: Dynamic Receipt (app/receipt/[id]/page.jsx)

```
'use client';
import Link from 'next/link';

export default async function ReceiptPage({ params }) {
  const { id } = await params; // Next.js 15 compatible

  return (
    <div className="max-w-md mx-auto bg-white p-8 rounded shadow text-center mt-10">
      <div className="text-5xl mb-4">✔</div>
      <h2 className="text-2xl font-bold mb-2">Order Confirmed</h2>
      <p className="text-gray-600 mb-4">Reference: <span className="font-mono bg-gray-100 px-2 py-1 rounded">{id}</span></p>
      <p className="mb-6 text-sm">Receipt saved locally. Thank you for using QuickPOS!</p>
      <Link href="/" className="inline-block bg-indigo-600 text-white px-6 py-2 rounded hover:bg-indigo-700">
        New Order
      </Link>
    </div>
  );
}
```

✔ Success Criteria:

- ☐ Checkout validates form in real-time
- ☐ Clears cart & redirects on success
- ☐ Dynamic [id] route displays order reference
- ☐ No Context used anywhere

Guidelines & Best Practices

Must-Have Implementation Rules

Requirement	How to Verify
No Context API	Search codebase: <code>createContext</code> , <code>useContext</code> → 0 matches
App Router Structure	<code>app/</code> directory used, <code>page.jsx</code> in each route
<code>useState</code> Used For	<code>search</code> , <code>form</code> , <code>items</code> , <code>processing</code> , <code>filtered</code>
<code>useEffect</code> Used For	<code>localStorage</code> sync, search filtering, form validation, empty cart redirect
Dynamic Routing	<code>/receipt/[id]/page.jsx</code> receives <code>async params</code>
Immutability	No <code>.push()</code> , <code>.splice()</code> , or direct state mutation in cart functions
Prop Drilling Explicit	State/functions passed down via props from <code>page.jsx</code> → components

Anti-Patterns to Avoid

```
// ❌ BAD: Mutating state directly
cart.items.push(newItem); // Never do this

// ✅ GOOD: Return new array
setItems(prev => [...prev, newItem]);

// ❌ BAD: useEffect missing dependencies
useEffect(() => {
  localStorage.setItem('pos-cart', JSON.stringify(items));
}); // Missing [items] → sync breaks

// ✅ GOOD
useEffect(() => {
  localStorage.setItem('pos-cart', JSON.stringify(items));
}, [items]);
```

Verification Checklist

- ☐ `npm run dev` starts cleanly
- ☐ Adding products updates cart instantly
- ☐ Refreshing page restores cart from `localStorage`
- ☐ Search filters without breaking layout
- ☐ Checkout button disabled until form valid
- ☐ Submission redirects to `/receipt/POS-XXXXX`
- ☐ ESLint passes with `react-hooks` rules
- ☐ Zero `createContext` or `useContext` usage



Submission Guidelines



What to Submit

1. **GitHub Repository** containing:
 - Complete `app/`, `components/`, `hooks/`, `lib/` folders
 - `README.md` with:
 - Setup instructions (`npm install && npm run dev`)
 - 3 screenshots (Catalog, Checkout, Receipt)
 - 1-paragraph reflection: *"How did prop drilling feel? Where did Context would help tomorrow?"*
2. **Clean Commit History** (minimum 5 commits: setup, hook, UI, routing, polish)
3. **ESLint Config** with `react-hooks` plugin enabled



Evaluation Criteria

Criteria	Points
✓ Correct App Router & dynamic routing	25
✓ <code>useState</code> & <code>useEffect</code> used appropriately	25
✓ Custom hook with <code>localStorage</code> sync	20
✓ Prop drilling implemented cleanly	15
✓ Form validation & routing flow	10
✓ Documentation & reflection	5
Total	100

💬 Instructor Note:

Today you felt the friction of prop drilling. Tomorrow, we remove it with Context API. But don't skip this step. Understanding *why* we lift state, *how* props flow, and *when* side effects should run is what separates tutorial followers from production engineers.

Push your repo, tag `#MERNBootcamp-POS`, and I'll review your hook architecture and dependency arrays. Keep your state predictable, your effects intentional, and your components focused.

— Kashif Raza | Clean props, clear routes, ready for Context tomorrow 🛒